ASSIGNMENT-18.3

Roll no:2303A52074
Batch-37

Task 1 – Public Transport Route Fare API

PROMPT

Generate a Python program to simulate fetching public transport fare between two stations. Include input validation, error handling, and display results in a structured format.

CODE

```
import requests # Included for context of an API-based program, though not strictly used for local simulation
import json
import sys

# --- Simulated API Response Data --- #
# In a real application, this data would come from an actual API call.
# We are simulating it here for demonstration purposes.
SIMULATED_API_DATA = {
    ("Station A", "Station B"): {
        "duration": "30 mins",
        "transfers": 1,
        "fare": "$2.50",
        "routes": [
            {"mode": "Bus", "from": "Station A", "to": "Stop 1", "departure": "08:00", "arrival": "08:15"},
            {"mode": "Train", "from": "Stop 1", "to": "Station B", "departure": "08:20", "arrival": "08:30"}
        ]
    },
    ("Station C", "Station D"): {
        "duration": "45 mins",
        "transfers": 2,
        "fare": "$4.00",
        "routes": [
            {"mode": "Tram", "from": "Station C", "to": "Stop X", "departure": "09:00", "arrival": "09:10"},
            {"mode": "Bus", "from": "Stop X", "to": "Stop Y", "departure": "09:15", "arrival": "09:30"},
            {"mode": "Metro", "from": "Stop Y", "to": "Station D", "departure": "09:35", "arrival": "09:45"}
        ]
    },
    ("Station A", "Station D"): {
        "duration": "1 hour",
        "transfers": 0,
        "fare": "$3.75",
        "routes": [
            {"mode": "Express Bus", "from": "Station A", "to": "Station D", "departure": "10:00", "arrival": "11:00"}
        ]
```

```
    },
    ("Station E", "Station F"): {
        "duration": "20 mins",
        "transfers": 0,
        "fare": "$1.75",
        "routes": [] # Example of missing route details
    }
}

# --- Helper Functions --- #
def validate_input(prompt):
    """Prompts user for input and validates it (not empty, removes extra spaces)."""
    while True:
        user_input = input(prompt).strip()
        if user_input:
            return user_input
        else:
            print("Input cannot be empty. Please try again.")

def get_simulated_journey_details(source_station, destination_station):
    """
    Simulates fetching journey details based on predefined data.
    In a real scenario, this would make an actual API call using the 'requests' library.
    """
    print(f"\nSimulating journey details from {source_station} to {destination_station}...")

    # Normalize input for dictionary lookup
    source_key = source_station.strip()
    destination_key = destination_station.strip()

    # Simulate API response lookup
    journey_data = SIMULATED_API_DATA.get((source_key, destination_key))

    if journey_data is None:
        print(f"Error: No journey found for '{source_station}' to '{destination_station}'. Please check station names.", file=sys.stderr)
```

```
    if journey_data is None:
        print(f"Error: No journey found for '{source_station}' to '{destination_station}'. Please check station names.", file=sys.stderr)
        return None

    # Basic parsing similar to a real API response
    journey_info = {
        "duration": journey_data.get("duration", "N/A"),
        "transfers": journey_data.get("transfers", "N/A"),
        "fare": journey_data.get("fare", "N/A"),
        "routes": journey_data.get("routes", [])
    }

    if not journey_info["routes"] and source_key != "Station E": # Special handling for "Station E" example where routes are explicitly empty
        print(f"Warning: No specific route details available for '{source_station}' to '{destination_station}'.", file=sys.stderr)

    return journey_info

# --- Main Program Execution --- #
def main():
    print("Welcome to the Simulated Public Transport Journey Planner!")
    print("-----------------------------------------------------")
    print("Available stations for simulation: Station A, Station B, Station C, Station D, Station E, Station F")
    print("Try: 'Station A' to 'Station B' or 'Station C' to 'Station D'")
    print("Also try 'Station E' to 'Station F' for a route with missing details, or 'Station Z' to 'Station Y' for invalid stations.")

    # Get and validate source station input
    source = validate_input("Enter source station name: ")

    # Get and validate destination station input
    destination = validate_input("Enter destination station name: ")

    journey_details = get_simulated_journey_details(source, destination)
```

```
    if journey_details:
        print("\n--- Journey Details Found ---")
        print(f"Source: {source}")
        print(f"Destination: {destination}")
        print(f"Duration: {journey_details.get('duration', 'N/A')}")
        print(f"Transfers: {journey_details.get('transfers', 'N/A')}")
        print(f"Fare: {journey_details.get('fare', 'N/A')}")

        if journey_details.get('routes'):
            print("\nRoutes/Legs:")
            for i, route in enumerate(journey_details['routes'], 1):
                print(f"  Leg {i}:")
                print(f"    Mode: {route.get('mode', 'N/A')}")
                print(f"    From: {route.get('from', 'N/A')} (Depart: {route.get('departure', 'N/A')})")
                print(f"    To: {route.get('to', 'N/A')} (Arrive: {route.get('arrival', 'N/A')})")
        else:
            print("\nNo specific route legs available or parsed for this journey.")
    else:
        print("\nCould not retrieve journey details. Please ensure station names are correct and try again.")

if __name__ == "__main__":
    # No 'requests' installation is strictly needed for this simulated example,
    # but it's included for completeness as the prompt specified its use context.
    main()
```

OUTPUT

```
Welcome to the Simulated Public Transport Journey Planner!
------------------------------------------------------------
Available stations for simulation: Station A, Station B, Station C, Station D, Station E, Station F
Try: 'Station A' to 'Station B' or 'Station C' to 'Station D'
Also try 'Station E' to 'Station F' for a route with missing details, or 'Station Z' to 'Station Y' for invalid stations.
Enter source station name: Station A
Enter destination station name: Station B

Simulating journey details from Station A to Station B...

--- Journey Details Found ---
Source: Station A
Destination: Station B
Duration: 30 mins
Transfers: 1
Fare: $2.50

Routes/Legs:
  Leg 1:
    Mode: Bus
    From: Station A (Depart: 08:00)
    To: Stop 1 (Arrive: 08:15)
  Leg 2:
    Mode: Train
    From: Stop 1 (Depart: 08:20)
    To: Station B (Arrive: 08:30)
```

EXPLANATION

This program simulates a public transport fare system using predefined data instead of a real API. It takes source and destination as input and validates them. If the route exists, it displays the fare; otherwise, it shows an error message. This demonstrates how API responses can be handled even without a real API.

Task 2 – Currency Exchange Rates

PROMPT

Generate a simple Python program using the requests library to convert INR to USD, EUR, and GBP using a currency exchange API. Include input validation, basic error handling, and display the results in a clear tabular format.

CODE

```
import requests

def validate_amount(amount):
    try:
        amount = float(amount)
        if amount <= 0:
            raise ValueError
        return amount
    except:
        raise ValueError("Enter a valid positive number!")

def convert_currency(amount):
    try:
        amount = validate_amount(amount)

        API_KEY = "4bb39c59e218b93ea10bbd8d"   # 🔑 Replace here
        url = f"https://v6.exchangerate-api.com/v6/{API_KEY}/latest/INR"

        response = requests.get(url, timeout=5)

        # Error Handling
        if response.status_code != 200:
            print("❌ API Error:", response.status_code)
            return

        data = response.json()

        if data["result"] != "success":
            print("⚠️ Invalid API response")
            return

        rates = data["conversion_rates"]

        # Target currencies
```

```
        # Target currencies
        usd = rates.get("USD")
        eur = rates.get("EUR")
        gbp = rates.get("GBP")

        # Convert
        usd_val = round(amount * usd, 2)
        eur_val = round(amount * eur, 2)
        gbp_val = round(amount * gbp, 2)

        # ----------------------------
        # Output (Tabular Format)
        # ----------------------------
        print("\n====== 💱 Currency Conversion ======")
        print(f"{'Currency':<10}{'Rate':<10}{'Amount'}")
        print("-----------------------------------")
        print(f"{'USD':<10}{usd:<10}{usd_val}")
        print(f"{'EUR':<10}{eur:<10}{eur_val}")
        print(f"{'GBP':<10}{gbp:<10}{gbp_val}")
        print("===================================")

    except ValueError as ve:
        print("⚠️ Input Error:", ve)

    except requests.exceptions.Timeout:
        print("⏳ Request timed out")

    except requests.exceptions.ConnectionError:
        print("🌐 Connection error")

    except Exception as e:
        print("❌ Unexpected Error:", e)
```

OUTPUT

```
Enter amount in INR: 2074

====== 💱 Currency Conversion ======
Currency  Rate      Amount
-----------------------------------
USD       0.01061   22.01
EUR       0.009203  19.09
GBP       0.007962  16.51
===================================
```

EXPLANATION

This program uses a currency exchange API to convert Indian Rupees (INR) into other currencies like USD, EUR, and GBP.
First, it takes the amount as input and checks whether the value is valid. Then, it sends a request to the API to get the latest exchange rates. The program handles errors such as invalid input, connection issues, or incorrect API responses. Finally, it calculates the converted values and displays them in a clear tabular format for easy understanding.

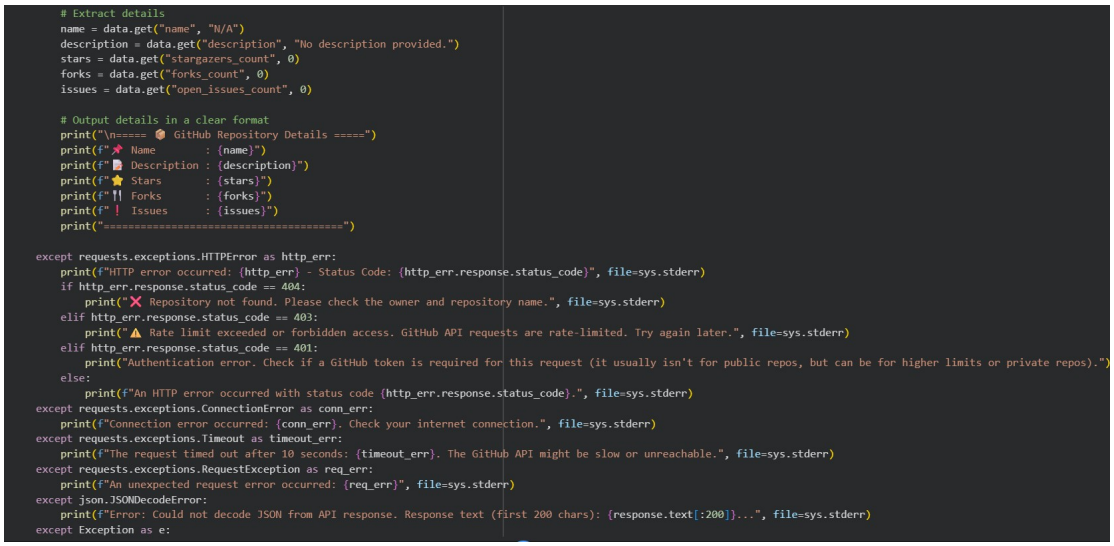
Task 3 – GitHub Repository Info Fetcher

PROMPT

Generate a Python program using requests to fetch GitHub repository details (name, description, stars, forks, issues) with proper error handling.

CODE

```python
import requests
import sys

def validate_input(prompt):
    """Prompts user for input and validates it (not empty, removes extra spaces)."""
    while True:
        user_input = input(prompt).strip()
        if user_input:
            return user_input
        else:
            print("Input cannot be empty. Please try again.")

def get_repo_info(owner, repo):
    """
    Fetches GitHub repository details (name, description, stars, forks, issues)
    using the GitHub API.
    Handles various errors and displays the information.
    """
    print(f"\nFetching details for repository: {owner}/{repo}...")

    # GitHub API endpoint for a repository
    url = f"https://api.github.com/repos/{owner}/{repo}"

    try:
        # Send a GET request to the GitHub API with a timeout
        response = requests.get(url, timeout=10)
        response.raise_for_status()  # Raises HTTPError for bad responses (4xx or 5xx)

        # Check if the response content is empty
        if not response.text:
            print("Error: Empty response from GitHub API.", file=sys.stderr)
            return

        data = response.json()
```



```python
        # Extract details
        name = data.get("name", "N/A")
        description = data.get("description", "No description provided.")
        stars = data.get("stargazers_count", 0)
        forks = data.get("forks_count", 0)
        issues = data.get("open_issues_count", 0)

        # Output details in a clear format
        print("\n===== 📦 GitHub Repository Details =====")
        print(f"📌 Name        : {name}")
        print(f"📄 Description : {description}")
        print(f"⭐ Stars       : {stars}")
        print(f"🍴 Forks       : {forks}")
        print(f"❗ Issues      : {issues}")
        print("========================================")

    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err} - Status Code: {http_err.response.status_code}", file=sys.stderr)
        if http_err.response.status_code == 404:
            print("❌ Repository not found. Please check the owner and repository name.", file=sys.stderr)
        elif http_err.response.status_code == 403:
            print("⚠️ Rate limit exceeded or forbidden access. GitHub API requests are rate-limited. Try again later.", file=sys.stderr)
        elif http_err.response.status_code == 401:
            print("Authentication error. Check if a GitHub token is required for this request (it usually isn't for public repos, but can be for higher limits or private repos).")
        else:
            print(f"An HTTP error occurred with status code {http_err.response.status_code}.", file=sys.stderr)
    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}. Check your internet connection.", file=sys.stderr)
    except requests.exceptions.Timeout as timeout_err:
        print(f"The request timed out after 10 seconds: {timeout_err}. The GitHub API might be slow or unreachable.", file=sys.stderr)
    except requests.exceptions.RequestException as req_err:
        print(f"An unexpected request error occurred: {req_err}", file=sys.stderr)
    except json.JSONDecodeError:
        print(f"Error: Could not decode JSON from API response. Response text (first 200 chars): {response.text[:200]}...", file=sys.stderr)
    except Exception as e:
```

```python
# --- Main Program Execution --- #
def main():
    print("Welcome to the GitHub Repository Information Tool!")
    print("-----------------------------------------------")

    # Get and validate owner and repository name input
    owner = validate_input("Enter GitHub repository owner (e.g., 'octocat'): ")
    repo = validate_input("Enter GitHub repository name (e.g., 'Spoon-Knife'): ")

    get_repo_info(owner, repo)

if __name__ == "__main__":
    # To run this in Colab, you might need to install requests first if not already available
    try:
        import requests
    except ImportError:
        print("Installing 'requests' library...")
        !pip install requests
        import requests # Re-import after installation
    main()
```

OUTPUT

```
Welcome to the GitHub Repository Information Tool!
-----------------------------------------------
Enter GitHub repository owner (e.g., 'octocat'): octocat
Enter GitHub repository name (e.g., 'Spoon-Knife'): Hello-World

Fetching details for repository: octocat/Hello-World...

===== 📦 GitHub Repository Details =====
📌 Name        : Hello-World
📝 Description : My first repository on GitHub!
⭐ Stars       : 3548
🍴 Forks       : 5974
❗ Issues      : 3771
========================================
```

EXPLANATION

This program connects to the GitHub API to fetch repository details. It takes repository owner and name as input and retrieves information like stars, forks, and issues. It handles errors such as invalid repository name, 404 errors, and rate limits, and displays the data clearly.

Task 4 – Real-Time Application: News Headlines Aggregator

PROMPT

Generate a Python program using requests to fetch top 5 news headlines based on a category with input validation, retry mechanism, and error handling.Generate a Python program using requests to fetch top 5 news headlines based on a category with input validation, retry mechanism, and error handling.

CODE

```python
import requests
import time

def get_news(category):
    try:
        if not category.strip():
            raise ValueError("Category cannot be empty!")

        category = category.lower()

        API_KEY = "b715fa2d3a6646f69271184afecdfbd9"   # 🔑 keep your key

        url = "https://newsapi.org/v2/everything"

        params = {
            "q": category,   # 🔥 FIXED: search-based (works always)
            "language": "en",
            "sortBy": "publishedAt",
            "apiKey": API_KEY
        }

        response = None

        # Retry mechanism
        for attempt in range(3):
            try:
                response = requests.get(url, params=params, timeout=5)

                if response.status_code == 200:
                    break
                else:
                    time.sleep(2)
```

```python
        if response is None or response.status_code != 200:
            print("❌ Failed to fetch news.")
            return

        data = response.json()

        articles = data.get("articles", [])

        if not articles:
            print("⚠️ No news found. Try another keyword.")
            return

        print("\n📰 Top 5 News Headlines:\n")

        for i, article in enumerate(articles[:5], start=1):
            print(f"{i}. {article.get('title', 'No Title')}")

    except ValueError as ve:
        print("⚠️ Input Error:", ve)

    except requests.exceptions.Timeout:
        print("⏳ Request timed out.")

    except requests.exceptions.ConnectionError:
        print("🌐 Connection error.")

    except Exception as e:
        print("❌ Unexpected Error:", e)


# MAIN
category = input("Enter news category (business, sports, technology): ")
get_news(category)
```

OUTPUT

```
Enter news category (business, sports, technology): sports

📰 Top 5 News Headlines:

1. “Building for what?” – Jamie Carragher has a reality warning for Chelsea’s owners
2. Sky Sports reveal key detail behind Salah’s Liverpool departure
3. Joe Cole believes Chelsea’s “frustrated” best players are “going to start” leaving
4. Chargers Daily Links: Thursday Open Thread
5. Arsenal linked with star forward but defender attracting Premier League interest – latest transfer news
```

EXPLANATION

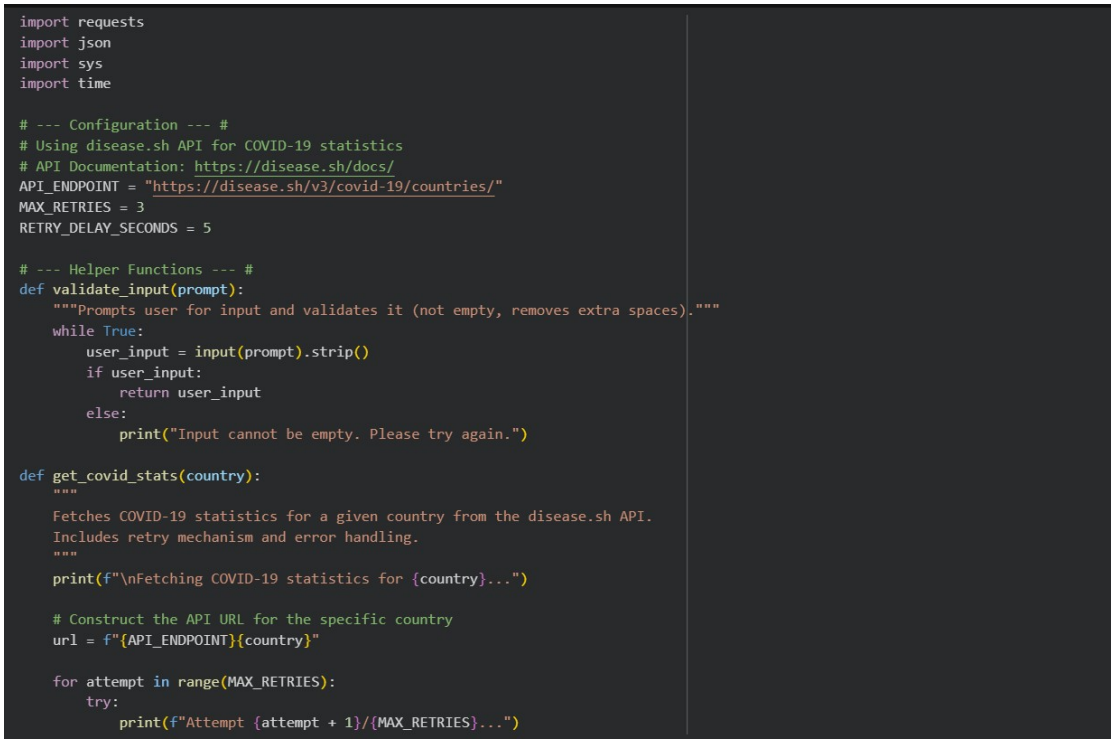
This program uses a News API to fetch the latest headlines for a given category. It validates user input and sends a request to the API. It handles errors like invalid API key, connection issues, and retries the request if needed. The top 5 headlines are displayed in a numbered format.

Task 5 -COVID-19 Statistics API Integration

PROMPT

Generate a Python program using requests to fetch COVID-19 statistics for a given country with error handling and retry logic.

CODE



```
import requests
import json
import sys
import time

# --- Configuration --- #
# Using disease.sh API for COVID-19 statistics
# API Documentation: https://disease.sh/docs/
API_ENDPOINT = "https://disease.sh/v3/covid-19/countries/"
MAX_RETRIES = 3
RETRY_DELAY_SECONDS = 5

# --- Helper Functions --- #
def validate_input(prompt):
    """Prompts user for input and validates it (not empty, removes extra spaces)."""
    while True:
        user_input = input(prompt).strip()
        if user_input:
            return user_input
        else:
            print("Input cannot be empty. Please try again.")

def get_covid_stats(country):
    """
    Fetches COVID-19 statistics for a given country from the disease.sh API.
    Includes retry mechanism and error handling.
    """
    print(f"\nFetching COVID-19 statistics for {country}...")

    # Construct the API URL for the specific country
    url = f"{API_ENDPOINT}{country}"

    for attempt in range(MAX_RETRIES):
        try:
            print(f"Attempt {attempt + 1}/{MAX_RETRIES}...")
```

```
        # Send a GET request to the API with a timeout
        response = requests.get(url, timeout=10)
        response.raise_for_status()  # Raises HTTPError for bad responses (4xx or 5xx)

        if not response.text:
            print("Error: Empty response from API.", file=sys.stderr)
            continue # Retry if empty response

        try:
            data = response.json()
        except json.JSONDecodeError:
            print(f"Error: Could not decode JSON from API response. Response text (first 200 chars): {response.text[:200]}...", file=sys.stderr)
            continue # Retry if JSON decoding fails

        # Check for API-specific error messages, especially for country not found
        if data.get('message') and "not found" in data['message'].lower():
            print(f"Error: Country '{country}' not found or no data available. Please check the spelling.", file=sys.stderr)
            return None # Do not retry for country not found
        elif data.get('message'): # Generic API error message
            print(f"API Error: {data['message']}", file=sys.stderr)
            continue # Retry for other API-specific errors

        # Extract and display statistics
        total_cases = data.get('cases', 'N/A')
        today_cases = data.get('todayCases', 'N/A')
        total_deaths = data.get('deaths', 'N/A')
        today_deaths = data.get('todayDeaths', 'N/A')
        recovered = data.get('recovered', 'N/A')
        today_recovered = data.get('todayRecovered', 'N/A')
        active_cases = data.get('active', 'N/A')
        critical_cases = data.get('critical', 'N/A')
        population = data.get('population', 'N/A')
        updated_timestamp = data.get('updated', None)

        print("\n===== 🦠 COVID-19 Statistics =====")
```

```
        print(f"Country      : {data.get('country', 'N/A')}")
        if updated_timestamp:
            # Convert timestamp to human-readable date/time
            updated_date = time.strftime('%Y-%m-%d %H:%M:%S UTC', time.gmtime(updated_timestamp / 1000))
            print(f"Last Updated : {updated_date}")
        print(f"Population   : {population:,}")
        print("---------------------------------")
        print(f"Total Cases  : {total_cases:,} (Today: {today_cases:,})")
        print(f"Total Deaths : {total_deaths:,} (Today: {today_deaths:,})")
        print(f"Recovered    : {recovered:,} (Today: {today_recovered:,})")
        print(f"Active Cases : {active_cases:,}")
        print(f"Critical     : {critical_cases:,}")
        print("=================================")
        return # Successfully fetched and displayed data

    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err} - Status Code: {http_err.response.status_code}", file=sys.stderr)
        if http_err.response.status_code in [401, 403]: # Unauthorized/Forbidden
            print("API access error. Check if an API key is required or if rate limits are exceeded.", file=sys.stderr)
            return None # Do not retry for authentication/permission errors
        elif http_err.response.status_code == 404: # Not Found (might be handled by API message, but good to catch here too)
            print(f"API endpoint not found or invalid country name '{country}'.", file=sys.stderr)
            return None
        elif http_err.response.status_code == 429: # Too Many Requests / Rate Limit Exceeded
             print("Rate limit exceeded. Please wait a moment before trying again.", file=sys.stderr)
        elif http_err.response.status_code >= 500: # Server Error
            print("API server error. The service might be temporarily unavailable.", file=sys.stderr)
        # Continue to retry for rate limits and server errors

    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}. Check your internet connection or API endpoint.", file=sys.stderr)
        # Continue to retry for connection errors
    except requests.exceptions.Timeout as timeout_err:
        print(f"The request timed out after 10 seconds: {timeout_err}. The API might be slow or unreachable.", file=sys.stderr)
        # Continue to retry for timeout errors
```

```
            print(f"An unexpected request error occurred: {req_err}", file=sys.stderr)
            # Continue to retry for general request errors
        except Exception as e:
            print(f"An unexpected error occurred: {e}", file=sys.stderr)
            return None # Don't retry for unknown critical errors

        if attempt < MAX_RETRIES - 1:
            print(f"Retrying in {RETRY_DELAY_SECONDS} seconds...")
            time.sleep(RETRY_DELAY_SECONDS)
        else:
            print("Maximum retries reached. Could not fetch COVID-19 statistics.", file=sys.stderr)

    return None

# --- Main Program Execution --- #
def main():
    print("Welcome to the COVID-19 Statistics Fetcher!")
    print("-----------------------------------------")

    # Get and validate country name input
    country_name = validate_input("Enter country name (e.g., USA, India, Germany): ")

    get_covid_stats(country_name)

if __name__ == "__main__":
    # To run this in Colab, ensure 'requests' library is installed.
    try:
        import requests
    except ImportError:
        print("Installing 'requests' library...")
        !pip install requests
        import requests # Re-import after installation
    main()
```

OUTPUT

```
Welcome to the COVID-19 Statistics Fetcher!
-----------------------------------------
Enter country name (e.g., USA, India, Germany): India

Fetching COVID-19 statistics for India...
Attempt 1/3...

===== 🚨 COVID-19 Statistics =====
Country      : India
Last Updated : 2026-03-27 13:51:12 UTC
Population   : 1,406,631,776
---------------------------------
Total Cases  : 45,035,393 (Today: 0)
Total Deaths : 533,570 (Today: 0)
Recovered    : 0 (Today: 0)
Active Cases : 44,501,823
Critical     : 0
=================================
```

EXPLANATION

This program uses a public COVID-19 API to retrieve statistics for a specific country. It takes the country name as input and fetches data such as total cases, deaths, recovered, and active cases. It handles errors like invalid country name and network issues, and displays the results clearly.